Unit-3: Forms and Hooks in React.JS

3.1 Forms: (Adding forms, Handling forms, Submitting forms)

Forms are an integral part of any modern web application. It allows the users to interact with the application as well as gather information from the users. Forms can perform many tasks that depend on the nature of your business requirements and logic such as authentication of the user, adding user, searching, filtering, booking, ordering, etc. A form can contain text fields, buttons, checkbox, radio button, etc.

Adding Forms in React

You add a form with React like any other element:

```
import React from 'react';
import ReactDOM from 'react-dom/client';


function MyForm() {
 return (
  <form>
   <label>Enter your name:
    <input type="text" />
   </label>
  </form>
 )
}
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<MyForm />);
```

Output:

Enter your name:

Handling Forms

- Handling forms is about how you handle the data when it changes value or gets submitted.
- In HTML, form data is usually handled by the DOM.
- In React, form data is usually handled by the components.
- When the data is handled by the components, all the data is stored in the component state.
- You can control changes by adding event handlers in the onChange attribute.
- We can use the useState Hook to keep track of each inputs value and provide a "single source of truth" for the entire application.

Example:

```
import { useState } from "react";
import ReactDOM from 'react-dom/client';

function MyForm() {

 const [name, setName] = useState("");


 return (
  <form>
   <label>Enter your name:
    <input
     type="text"
     value={name}
     onChange={(e) => setName(e.target.value)}
    />
   </label>
  </form>
 )
```

```
}

const root = ReactDOM.createRoot(document.getElementById('root'));

root.render(<MyForm />);
```

Output:

Enter your name:

Submitting Forms

You can control the submit action by adding an event handler in the onSubmit attribute for the <form> :

Example:

Add a submit button and an event handler in the onSubmit attribute:

```
import { useState } from "react";
import ReactDOM from 'react-dom/client';

function MyForm() {
 const [name, setName] = useState("");

 const handleSubmit = (event) => {
  event.preventDefault();
  alert(`The name you entered was: ${name}`);
 }

 return (
  <form onSubmit={handleSubmit}>
   <label>Enter your name:
    <input
     type="text"
     value={name}
     onChange={(e) => setName(e.target.value)}
    />
   </label>
   <input type="submit" />
  </form>
 )
```

```
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<MyForm />);
```

3.1.1 event.target.name and event. Target.event, React Memo

- You can control the values of more than one input field by adding a name attribute to each element.
- We will initialize our state with an empty object.
- To access the fields in the event handler use the event.target.name and event.target.value syntax.
- To update the state, use square brackets [bracket notation] around the property name.

Example:

```
import { useState } from 'react';
import ReactDOM from 'react-dom/client';


function MyForm() {
 const [inputs, setInputs] = useState({});


 const handleChange = (event) => {
  const name = event.target.name;
  const value = event.target.value;
  setInputs(values => ({...values, [name]: value}))
 }
 const handleSubmit = (event) => {
  event.preventDefault();
  alert(inputs);
 }
```

```
 return (
   <form onSubmit={handleSubmit}>
     <label>Enter your name:
     <input
       type="text"
       name="username"
       value={inputs.username || ""}
       onChange={handleChange}
     />
     </label>
     <label>Enter your age:
       <input
         type="number"
         name="age"
         value={inputs.age || ""}
         onChange={handleChange}
       />
       </label>
       <input type="submit" />
   </form>
 )
}


const root = ReactDOM.createRoot(document.getElementById('root'));

root.render(<MyForm />);
```

3.1.2 Components (TextArea, Drop down list (SELECT))

1. **TextArea**

- The textarea element in React is slightly different from ordinary HTML.
- In HTML the value of a textarea was the text between the start tag <textarea> and the end tag </textarea>.

```
<textarea>
        Content of the textarea.
</textarea>
```

In React the value of a textarea is placed in a value attribute. We'll use the useState Hook to manage the value of the textarea:

Example:

```
import { useState } from 'react';
import ReactDOM from 'react-dom/client';

function MyForm() {
 const [textarea, setTextarea] = useState(
  "The content of a textarea goes in the value attribute"
 );
 const handleChange = (event) => {
  setTextarea(event.target.value)
 }
 return (
  <form>
   <textarea value={textarea} onChange={handleChange} />
  </form>
```

```
 )
}
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<MyForm />);
```

2. **Drop down list (SELECT)**

   A drop down list, or a select box, in React is also a bit different from HTML.

   in HTML, the selected value in the drop down list was defined with the selected attribute:

HTML:

```
<select>
  <option value="Ford">Ford</option>
  <option value="Volvo" selected>Volvo</option>
  <option value="Fiat">Fiat</option>
</select>
```

Example:

```
import { useState } from "react";
import ReactDOM from "react-dom/client";

function MyForm() {
  const [myCar, setMyCar] = useState("Volvo");

  const handleChange = (event) => {
    setMyCar(event.target.value)
  }
```

```
return (
  <form>
   <select value={myCar} onChange={handleChange}>
    <option value="Ford">Ford</option>
    <option value="Volvo">Volvo</option>
    <option value="Fiat">Fiat</option>
   </select>
  </form>
 )
}


const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<MyForm />);
```

3.2 Hooks:

What are Hooks?

Hooks are used to give functional components an access to use the states and are used to manage side-effects in React. They were introduced React 16.8. They let developers use state and other React features without writing a class For example- State of a component It is important to note that hooks are not used inside the classes.

When to use a Hooks?

If you write a function component, and then you want to add some state to it, previously you do this by converting it to a class. But, now you can do it by using a Hook inside the existing function component.

Rules of Hooks

Hooks are similar to JavaScript functions, but you need to follow these two rules when using them. Hooks rule ensures that all the stateful logic in a component is visible in its source code. These rules are:

- Only functional components can use hooks
- Calling of hooks should always be done at top level of components
- Hooks should not be inside conditional statements

Advantages of Using React Hooks

1. Simplified Code

One of the primary benefits of using React Hooks is that it simplifies the codebase. Hooks eliminate the need for class components, which often require more boilerplate code and can be harder to read and understand. With Hooks, developers can write cleaner, more intuitive code.

2. Improved Reusability

React Hooks make it easier to reuse stateful logic across components. With custom hooks, developers can extract component logic into reusable functions. This promotes cleaner, more modular code, and reduces duplication.

3. Easier Testing and Debugging

Functional components that use Hooks are generally easier to test and debug than class components. Since Hooks promote separation of concerns and a more functional programming style, developers can write more predictable and testable code.

4. Reduced Bundle Size

By using functional components with Hooks instead of class components, developers can reduce the overall size of their application bundle. This can lead to faster load times and improved performance for users.

3.2.1 useState, useEffect, useContext

1. useState()

Hook state is the new way of declaring a state in React app. Hook uses useState() functional component for setting and retrieving state. It should be noted that one use of useState() can only be used to declare one state variable. It was introduced in version 16.8.

Let us understand Hook state with the following example.

Example

```
import React, { useState } from 'react';

function App() {
   const [click, setClick] = useState(0);
   // using array destructuring here
   // to assign initial value 0
   // to click and a reference to the function
   // that updates click to setClick
   return (
      <div>
         <p>You clicked {click} times</p>

         <button onClick={() => setClick(click + 1)}>
            Click me
         </button>
      </div>
   );
}

export default App;
```

2. useEffect

- The useEffect Hook allows you to perform side effects in your components.
- Some examples of side effects are: fetching data, directly updating the DOM, and timers.
- useEffect accepts two arguments. The second argument is optional.
- useEffect(<function>, <dependency>)
- It does not use components lifecycle methods which are available in class components.
- Effects Hooks are equivalent to componentDidMount(), componentDidUpdate(), and componentWillUnmount() lifecycle methods.

Example

```
import React, { useState, useEffect } from 'react';

function CounterExample() {
 const [count, setCount] = useState(0);

 // Similar to componentDidMount and componentDidUpdate:
 useEffect(() => {
  // Update the document title using the browser API
  document.title = `You clicked ${count} times`;
 });

 return (
  <div>
   <p>You clicked {count} times</p>
   <button onClick={() => setCount(count + 1)}>
    Click me
   </button>
  </div>
 );
}
export default CounterExample;
```

3. useContext()

It is use with use State hook to share state between deeply nested components.

```
import { useState, createContext, useContext } from "react";
import ReactDOM from "react-dom/client";

const UserContext = createContext();

function Component1() {
 const [user, setUser] = useState("Jesse Hall");

 return (
  <UserContext.Provider value={user}>
   <h1>{`Hello ${user}!`}</h1>
   <Component2 />
  </UserContext.Provider>
 );
}

function Component2() {
 return (
  <>
   <h1>Component 2</h1>
   <Component3 />
  </>
```

```
  );
}

function Component3() {
  return (
    <>
      <h1>Component 3</h1>
      <Component4 />
    </>
  );
}

function Component4() {
  return (
    <>
      <h1>Component 4</h1>
      <Component5 />
    </>
  );
}

function Component5() {
  const user = useContext(UserContext);

  return (
```

```
    <>
      <h1>Component 5</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Component1 />);
```

3.2.2 useRef, useReducer, useCallback, useMemo

1. useRef()

- The useRef Hook allows you to persist values between renders.
- It can be used to store a mutable value that does not cause a re-render when updated.
- It can be used to access a DOM element directly.
- useRef() only returns one item.

Example:

```
import { useRef } from "react";
import ReactDOM from "react-dom/client";

function App() {
  const inputElement = useRef();

  const focusInput = () => {
```

```
    inputElement.current.focus();
  };

  return (
    <>
      <input type="text" ref={inputElement} />
      <button onClick={focusInput}>Focus Input</button>
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

2. useReducer()

- The useReducer Hook is similar to the useState Hook.
- It allows for custom state logic.
- If you find yourself keeping track of multiple pieces of state that rely on complex logic, useReducer may be useful.

Syntax

```
useReducer(<reducer>, <initialState>)
```

Example:

```
import { useReducer } from 'react';

function reducer(state, action) {
  if (action.type === 'incremented_age') {
    return {
      age: state.age + 1
```

```
    };
  }
  throw Error('Unknown action.');
}

export default function Counter() {
  const [state, dispatch] = useReducer(reducer, { age: 42 });

  return (
    <>
      <button onClick={() => {
        dispatch({ type: 'incremented_age' })
      }}>
        Increment age
      </button>
      <p>Hello! You are {state.age}.</p>
    </>
  );
}
```

3. **useCallback ()**

- This allows us to isolate resource intensive functions so that they will not automatically run on every render.
- The useCallback Hook only runs when one of its dependencies update.
- This can improve performance.

Example:

```
import { useCallback } from 'react';



export default function ProductPage({ productId, referrer, theme }) {

  const handleSubmit = useCallback((orderDetails) => {

    post('/product/' + productId + '/buy', {

      referrer,
```

```
    orderDetails,
  });
}, [productId, referrer]);
```

4. useMemo()

- The useMemo and useCallback Hooks are similar. The main difference is that useMemo returns a memoized value and useCallback returns a memoized function.
- The React useMemo Hook returns a memoized value.
- The `useMemo` Hook only runs when one of its dependencies update.

Example:

```
import { useMemo } from 'react';

function TodoList({ todos, tab }) {
  const visibleTodos = useMemo(
    () => filterTodos(todos, tab),
    [todos, tab]
  );
  // ...
}
```

3.2.3 Hook: Building custom hook, advantages and use

React Custom Hooks Build a Hook

- Hooks are reusable functions.
- When you have component logic that needs to be used by multiple components, we can extract that logic to a custom Hook.
- Custom Hooks start with "use". Example: useFetch.
- Building custom Hooks allows you to extract component logic into reusable functions.

Example:

```
import React, { useState, useEffect } from 'react';

const useDocumentTitle = title => {
 useEffect(() => {
  document.title = title;
 }, [title])
}

function CustomCounter() {
 const [count, setCount] = useState(0);
 const incrementCount = () => setCount(count + 1);
 useDocumentTitle(`You clicked ${count} times`);
 // useEffect(() => {
 //   document.title = `You clicked ${count} times`
 // });

 return (
  <div>
   <p>You clicked {count} times</p>
   <button onClick={incrementCount}>Click me</button>
  </div>
 )
}
export default CustomCounter;
```

Advantages of custom Hook

1. **Reusability**
   Custom React JS hooks offer reusability as when a custom hook is created, it can be reused easily, which ensures clean code and reduces the time to write the code. It also enhances the rendering speed of the code as a custom hook does not need to be rendered again and again while rendering the whole code.

2. **Readability**
   Instead of using High-Order Components (HOCs), one can use custom hooks to improve the readability of the code. Complex codes can become hard to read if layers of providers surround the components, consumers, HOCs, render props, and other abstractions, generally known as wrapper hell. On the other hand, using custom React JS hooks can provide cleaner logic and a better way to understand the relationship between data and the component.

3. **Testability**
   Generally, the test containers and the presentational components are tested separately in React. This is not a trouble when it comes to unit tests. But, if a container contains several HOCs, it becomes difficult as you will have to test the containers and the components together to do the integration tests. Using custom React JS hooks, this problem can be eliminated as the hooks allow you to combine containers and components into one component. It also makes it easier to write separate unit tests for custom hooks. Using custom hooks also makes it easier to mock hooks when compared to mock HOCs as it is similar to mocking a function.

Example:1 Create a form in React.js with Language Selection label, Dropdown List Which contains different Languages, and submit button Components.

```
import React, { useState } from 'react';

const languageOptions = [

 'English',

 'Hindi',

 'Gujarati',

 'Punjabi',
```

```
  'Tamil',
  'Sanskrit'
];

const LanguageForm = () => {
  const [selectedLanguage, setSelectedLanguage] = useState('');

  const handleLanguageChange = (event) => {
    setSelectedLanguage(event.target.value);
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    alert(`Selected language: ${selectedLanguage}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <label htmlFor="languageSelection">Language Selection:</label>
<select id="languageSelection"  value={selectedLanguage}
onChange={handleLanguageChange}>
          <option value="">Select a language</option>
          {languageOptions.map((language, index) => (
            <option key={index} value={language}>
```

```
            {language}
          </option>
        ))}
      </select>
    </div>
    <button type="submit">Submit</button>
  </form>
 );
};
export default LanguageForm;

import React from 'react';
import LanguageForm from './LanguageForm'; // Assuming the file is in the same directory

const App = () => {
 return (
  <div>
   <h1>Language Selection Form</h1>
   <LanguageForm />
  </div>
 );
};

export default App;
```

Example: 2

Create a simple counter using React.js which increments or decrements count dynamically On-screen as the user clicks on buttons. Use following steps

1. Prepare two HTML buttons, one for increment value another for decrement value.
2. If we click on increment button then value will be increased by 1.. such 1,2,3,4…. And
   If we click on decrement button then value will be decrease by 1.. such 4,3,2,1…. And
   So on…

Create Counter Component:

```
import React, { useState } from 'react';


const Counter = () => {
 const [count, setCount] = useState(0);


 const increment = () => {
  setCount(count + 1);
 };


 const decrement = () => {
  setCount(count - 1);
 };


 return (
  <div>
   <h1>Count: {count}</h1>
   <button onClick={increment}>Increment</button>
   <button onClick={decrement}>Decrement</button>
```

```
    </div>
  );
};

export default Counter;
```

Use the Counter Component in App:

```
import React from 'react';
import Counter from './Counter'; // Assuming the file is in the same directory

const App = () => {
  return (
    <div>
      <h1>Simple Counter App</h1>
      <Counter />
    </div>
  );
};

export default App;
```